Engineer 004: Real-Time Analytics Pipeline

Brief version: 2026-07 · Fixture sha256: `1aeb24b415009e89fcf8acb5a178410faf216dc17b16920d9849ecc8bbb24235` · Candidate: Edwin Plange

Written answer

The decision that shapes everything

The binding constraint is not the $50K budget. It is two engineers for six months. My sizing model (`sizing.py`) puts the whole system at about $11.2K a month [Estimated], 78% under the ceiling [Estimated]. So I spend money to buy operations away: managed services everywhere, and spike capacity left permanently on instead of clever autoscaling. Every choice below is "what can two engineers run at 3am".

1. Architecture and technology choices

Diagram: `architecture.mmd` (Mermaid, renders on GitHub).

Collector (ECS Fargate, behind the existing hostname). Resolves tenant from the write key, never from the payload. Stamps `received_at` server-side. Returns 200 only after Kinesis confirms the put. If Kinesis refuses after retries, it writes the batch to an S3 spool that a job replays. My hypothesis is that much of the current 3% loss [Assumed] (given in brief) lives at this hop, because an ack that precedes durability is how events vanish. Phase 0 tests that hypothesis rather than assuming it. If even the spool write fails, the collector returns 5xx and the SDK retries; evt-0002 shows the SDK already does this [Observed].

Kinesis Data Streams, provisioned, 23 shards always on [Estimated]. Rejected MSK: cheaper per GB at scale, but broker operations for two people, and at roughly $270 a month for Kinesis [Estimated] there is nothing to save. Partition key `tenant_id:anonymous_id` keeps one visitor's events ordered and spreads tenants.

Consumer (Fargate, KCL, Python). Parse, normalise legacy shapes, dedupe, redact PII, flag bots, apply the clock policy, route. Rejected Managed Flink: real windowing and exactly-once, but a new paradigm for a Python/Node team, and the MVP needs counters and rolling counts, not complex windows. It is my first upgrade if segment logic outgrows that.

Stores.

- **Redis** (already in the stack): dedupe keys, per-second live counters, and rolling per-user features such as pricing views in the last 24 hours. This serves the <5 second dashboards and personalisation triggers.
- **ClickHouse Cloud on AWS**: history and segmentation, using `ReplacingMergeTree` on `(tenant_id, event_id)` and tenant row policies. Rejected scaling Postgres: about 1.5 billion rows a month [Estimated] recreates today's failure. Redshift Serverless is my fallback if procurement requires AWS-native billing. I have not benchmarked either for this workload.
- **S3 raw Parquet via Firehose**: the replay source and the export path.

Identity. Events are stored against `anonymous_id`. A Postgres identity map (`tenant, anonymous_id, user_id, source_event_id`) resolves `user_id` at query time, so an `identify` never rewrites history. In the fixture [Observed], `evt-0008` retroactively attributes `evt-0004` and `evt-0005` to `u-2209` with zero writes.

Latency budget. The fixture shows an apparent client-to-server delay with a median of 163 ms (range 119 to 206 ms, n=19 clean events) [Observed] (computed by `detect_anomalies.py`, see `output/anomaly_report.md`). That figure mixes network transit with client clock offset, so it is a weak proxy, not a latency measurement. Adding put (~100 ms [Assumed]), consumer poll (≤1 s) and a 1 s dashboard push gives about 2.5 s at p50 [Estimated]. p99 is unproven until the week-2 load test. A canary SLO enforces it: p99 under 5 s.

2. Scale, reliability and migration

Throughput. 50M a day is 579 events/s on average. With a 3x busy hour [Assumed] that is 1,736/s, and a 10x spike on top gives 17,361/s [Estimated]. At 1 KB per event [Assumed] that is 17 MB/s, which needs 18 shards; with 1.3x headroom [Assumed], 23 [Estimated]. Keeping spike capacity on permanently costs about $250 a month [Estimated], so I don't rely on resharding during an unannounced spike. A stress run with 20x spikes and 2 KB events (`output/sizing_spike20_kb2.md`) still comes to about $13.8K a month [Estimated].

Guarantees, stated precisely. Delivery is at-least-once end to end. Counts are effectively-once through dedupe on `(tenant_id, event_id)`: a Redis `SET NX` with a 24-hour TTL [Assumed] in the hot path, and the ClickHouse merge as the backstop. "Zero loss" is not asserted; it is detected:

- **A count ledger** per tenant per minute at every hop: collector accepted, consumed, ClickHouse inserted, S3 written. Any gap pages someone.
- **Canary events** every 10 seconds [Assumed] per availability zone from a synthetic tenant. A canary missing after 60 seconds [Assumed] fires an alert.

Under overload, things degrade in this order:

1. Personalisation feature freshness.
2. Dashboard freshness.
3. Never event acceptance.

Consumer iterator age above 5 seconds [Assumed] alerts.

Migration (the SDK can't change, and integrations can't break). Week counts are [Estimated] for two engineers:

- **Phase 0 (weeks 1-2) [Estimated]:** the new collector goes in front as a pure proxy to legacy. Behaviour is identical. For the first time we measure legacy loss with the ledger instead of estimating it.
- **Phase 1 (weeks 3-8):** the collector tees every event to Kinesis. The new pipeline runs in shadow, with a parallel-run comparison per tenant, per hour and per event type.
- **Phase 2 (weeks 9-12, MVP):** dashboards move per tenant cohort behind a flag: internal, then 5%, 25%, 100%. Legacy stays fed throughout.
- **Phase 3 (months 4-6):** segments, exports and deletion automation. Legacy is decommissioned after 30 green days.

Rollback. Flip the cohort flag back to legacy dashboards, which takes minutes because legacy never stopped receiving data. For the collector itself, move the ALB weight back to the old endpoint.

Rollback triggers (thresholds [Assumed], tuned in Phase 1):

- Any canary loss within an hour [Assumed].
- p99 above 5 s for 15 minutes.
- Any tenant's parallel-run mismatch outside tolerance for 2 consecutive hours.

"Accuracy verified" is a test, not a feeling. It means 14 consecutive days [Assumed], including one 10x replay load test from S3 raw, with all of the following:

- Zero canary loss.
- Zero canary duplicates after dedupe.
- For every tenant and hour, new counts at or above legacy, with the gap explained by legacy loss in the ledger to within 0.1% [Assumed].

3. Tradeoffs and risks

Optimising for: no silent loss, operability by two people, and compliance built into the data layout.
Sacrificing:

- Exactly-once processing.
- Sub-second freshness.
- MVP scope: warehouse exports, the segment builder UI, automated deletion execution (a runbook in the MVP) and multi-region all wait until months 4-6 or later.

With more time or budget:

- Managed Flink for complex segments.
- Multi-region ingest.
- An SDK v2 with per-device sequence numbers, which would make loss exactly measurable instead of inferred.

Compliance as architecture (deadlines [Benchmarked] against the cited statutes):

- Tenant comes from the write key.
- PII is redacted before the first write anywhere.
- Deletion uses the identity map to find every `anonymous_id`, then deletes in Redis immediately, in ClickHouse in daily batches and in S3 raw in weekly partition rewrites, and sends deletion manifests downstream to exports.
- An audit log without PII records each request.
- Targets are within one month for GDPR [Benchmarked] (GDPR Art. 12(3)) and 45 days for CCPA [Benchmarked] (Cal. Civ. Code §1798.130).

4. What the fixture actually contains

`detect_anomalies.py` finds 13 anomaly classes across 25 lines and 23 unique parsed events [Observed] (`output/anomaly_report.md`). All 15 tests pass [Observed] (`output/test_run.txt`).

Class	event_id, all [Observed] in the fixture	Pipeline action
Duplicate redelivery	evt-0002 (lines 2 and 5, identical except `received_at`, 7,428 ms apart)	Dedupe on `(tenant_id, event_id)`
Minor clock skew	evt-0005 (client 47 s ahead)	Keep client ts, record skew
Major clock skew	evt-0006 (client 65 min ahead)	Event time = `received_at`; keep `client_ts`
Implausible ts	evt-0016 (year 2027)	Event time = `received_at`; flag device
PII in properties	evt-0007 (`contact_email`, `phone`)	Redact before storage; alert tenant
Legacy schema	evt-0009 (`pageview`, `timestamp`, `page_path`, `ref`)	Versioned adapter, never reject
Missing `received_at`	evt-0009	Collector stamps always; this path bypassed it
Missing tenant	evt-0011	Resolve from write key or quarantine
Malformed JSON	evt-0020 (missing closing brace)	Dead letter with raw bytes, counted
Bot burst	evt-0012 to evt-0015 (4 page views in 50 ms, scanner referrer)	Keep with `is_bot`; exclude from segments
Privacy request via SDK	evt-0017	Suppress now, delete after verification
Client-computed count	evt-0019	Recompute server-side
Retroactive stitch	evt-0001, 0002, 0004, 0005, 0006, 0007	Resolve at query time

What I would not act on at face value:

1. **evt-0019 claims `count_today: 3` pricing views.** Server-side, only one pricing page view (evt-0001) exists for that visitor in the sample. The sample may be a slice, so the claim is unverifiable rather than proven false. Either way, a "viewed pricing 3x" segment must count server-side, never trust the client.
2. **evt-0017 is a GDPR deletion arriving through the public SDK.** It is unauthenticated, and the only thing linking `anon-77a` (evt-0006) to `u-1077` is the request itself. Executing it blindly lets anyone delete another visitor's data. Suppression is reversible and safe, so apply it now. Deletion waits for the tenant to confirm through the authenticated API.
3. **evt-0002's second copy arrives 7.4 s later.** My hypothesis is an SDK retry after a slow ack, which would mean the current system manufactures duplicates under load. One example does not prove it, so Phase 0's ledger tests it.
4. **evt-0011 has no tenant.** Its `/blog/launch` path would tempt a guess. A wrong tenant guess is a data leak, so it gets quarantined.
5. **The sample itself.** 25 lines and 3 tenants [Observed]. I won't estimate bot share, skew rates or loss from it.

Operating artifact

Repository: https://github.com/JayPlange/beat-claude-eng004 (counts below [Observed] from the run logs)

- `detect_anomalies.py` is the consumer's validation pass, runnable on the fixture.
- `tests/test_detect.py` has 15 tests [Observed]: one per anomaly class on synthetic input, plus fixture regression checks.
- `sizing.py` is the capacity and cost model, with every input labelled and changeable.
- `architecture.mmd` is the diagram.
- `output/` holds the generated reports.

Evidence log

Claim	Tier	Proof ([Observed] where run, else [Estimated])
Detector catches every anomaly class in this fixture	Tier 3	`output/anomaly_report.md`, `output/test_run.txt`; reproduce with `python3 detect_anomalies.py fixtures/event_sample.jsonl`
Ingest delay median 163 ms in the sample	Tier 3	Same report, computed from fixture timestamps
Throughput and shard sizing	Tier 2	`sizing.py` arithmetic; inputs labelled
Total of about $11.2K a month	Tier 2 for the arithmetic, Tier 0 for unit prices	`output/sizing_base.md`; prices need AWS Pricing Calculator confirmation
p50 of about 2.5 s, p99 under 5 s	Tier 0	Design estimate; becomes Tier 3 after the week-2 load test
Zero loss	Tier 0	Design claim; proven only by Phase 1 canaries and the ledger

Number source labels

Every number above carries [Observed], [Estimated], [Benchmarked] or [Assumed]. The full input table with labels is in `sizing.py` and its generated `output/sizing_base.md`. Figures given in the brief are labelled [Assumed] (given in brief), because I did not measure them.

AI usage disclosure

I used Claude Code (Opus 5.5). It drafted `detect_anomalies.py`, the tests, `sizing.py`, the diagram and the first draft of this document, working from my direction on approach and constraints.

What I checked myself:

- I read every fixture row against the report and confirmed each cited event_id.
- I ran the tests.
- I re-derived the throughput arithmetic by hand.
- I changed the capacity strategy to always-on after the model showed resharding saved almost nothing.

Known weak spots:

- The AWS and ClickHouse unit prices are unverified.
- The bot rule is calibrated on one cluster.
- The PII detector is regex-based and will miss free-text PII.

What breaks it

- **Spikes beyond about 13x busy hour [Estimated]** exceed the 23 shards [Estimated]. Detect: write-throttle alarms. Response: the S3 spool absorbs the excess, and a reshard follows.
- **A single hot visitor or bot** concentrated on one partition key creates a hot shard. Response: per-key rate limiting at the collector.
- **SDK retries older than the 24-hour dedupe TTL**, such as offline devices, get through Redis. The ClickHouse merge catches them eventually, so exact queries need `FINAL`, which is slower.
- **A retried event whose time came from `received_at`** (major skew) gets a different event time on each copy. If the ClickHouse sort key includes event time, the merge will not collapse it. Fix: dedupe on first-seen in Redis and carry the first copy's event time.
- **The migration tee to legacy** must be asynchronous, or legacy slowness blocks new acks. The cost is that legacy may lose a little more during migration, which the parallel-run tolerance has to allow for.
- **Shared devices** map one `anonymous_id` to several users. The identity map must store a history, not a single value.
- **A leaked write key** lets someone spoof a tenant. Response: key rotation and per-key anomaly alerts.
- **Legacy shapes beyond evt-0009** are unknown. Unmapped shapes go to the dead letter, not to drop.
- **Event size or unit prices far from my assumptions** move the cost. Re-run `sizing.py`.

What stays human

- **Executing a deletion.** The tenant, as data controller, confirms it. Automation only suppresses.
- **Promoting a migration cohort or declaring rollback.** The on-call engineer decides from the ledger and the canaries.
- **Per-tenant PII policy:** hash or drop.
- **Changes to bot thresholds.** False positives hide real customers.
- **Adding a new legacy adapter.** A wrong mapping corrupts history silently.
- **Decommissioning the legacy pipeline.**

Artifact access

Public repo, no login: https://github.com/JayPlange/beat-claude-eng004. Standard library only; tested on Python 3.13 [Observed] (`output/test_run.txt`).

```
$ python3 detect_anomalies.py fixtures/event_sample.jsonl
$ python3 -m unittest discover -s tests -v
$ python3 sizing.py --spike 20 --kb 2
```